WEB-AGENT

Arquitectura de diseño, implementación existente y plan técnico de construcción

Documento técnico consolidado | Fecha: 10/10/2026

Agente generalista para operar aplicaciones legacy y escritorios remotos sin API, con visión como modalidad principal y automatización controlada.

Objetivo	Documentar la arquitectura observada, la arquitectura objetivo, los contratos, el modelo de ejecución, los límites de seguridad y las brechas que separan la base actual del sistema exigido.
Principio rector	El modelo propone; el núcleo propio valida, autoriza, ejecuta, verifica y registra. La inferencia no obtiene permisos ni herramientas de ejecución por sí misma.
Estado de evidencia	Basado en la descripción y revisión estática resumida en el archivo aportado. No equivale a ejecutar el repositorio, probar el sistema ni certificar un despliegue.

Cómo leer este documento

- **Implementado / existente:** componentes que el archivo fuente identifica en el código compartido; pueden tener limitaciones o defectos detallados aquí.
- **Parcial:** existe una base, pero la integración, la política o la verificación no cubren todo el objetivo.
- **Pendiente / propuesto:** arquitectura recomendada o requisito aún no demostrado como implementado.

Alcance técnico

Este documento no afirma que la arquitectura objetivo ya esté construida. Las conclusiones sobre código reflejan únicamente lo que la fuente aportada dice haber observado; no se dispone aquí de una ejecución reproducible ni de resultados de pruebas. Las reglas oficiales del reto y el README definitivo deben prevalecer si difieren de los ejemplos de diseño incluidos en el archivo.

Índice técnico

- 1. Resumen ejecutivo
- 2. Requisitos y fronteras del reto
- 3. Estado de implementación observado
- 4. Arquitectura modular objetivo
- 5. Percepción dual: visión y DOM
- 6. Modelo de observación y geometría
- 7. Contrato de acciones y contexto de ejecución
- 8. Runner, estados y ejecución determinista
- 9. Skills nativas
- 10. Política, autorización y fronteras de confianza
- 11. Red, sesiones y escritorios remotos
- 12. Verificación y recuperación
- 13. API REST, estados públicos y deadlines
- 14. Auditoría, evidencia y secretos
- 15. Inferencia, proveedores y presupuestos
- 16. Brechas y defectos específicos
- 17. Plan de construcción por fases
- 18. Estrategia de pruebas y criterios de aceptación
- 19. Decisiones de arquitectura (ADR)
- 20. Checklist operativo de salida

Términos clave

DOM: estructura del documento web; **GUI**: interfaz gráfica; **VLM**: modelo visión-lenguaje; **LLM**: modelo de lenguaje; **deadline monotónico**: límite calculado con reloj monotónico para evitar cambios por ajuste del reloj del sistema; **postcondición**: evidencia observable de que la operación produjo el resultado esperado.

1. Resumen ejecutivo

web-agent se describe como un agente de automatización con núcleo propio: esquemas tipados, perfiles de tarea, propuestas estructuradas, políticas, sesiones de Playwright, adaptadores de inferencia, un runner que coordina la ejecución y un sistema de auditoría redactada. Esta base es útil para construir el producto, pero no demuestra por sí sola operación visual autónoma ni cumplimiento del contrato REST externo.

La arquitectura objetivo convierte la observación visual de la aplicación en el modo principal, especialmente para escritorios remotos, interfaces canvas y aplicaciones legacy cuyo DOM no es accesible o no debe utilizarse. El adaptador DOM se conserva como opción cuando la política y el entorno lo permiten. La percepción produce evidencia; no autoriza acciones.

La distinción más importante es la separación entre inferencia y ejecución: el modelo devuelve una propuesta no confiable, el núcleo valida esquema y contexto, la política decide si la operación está permitida, el ejecutor realiza un conjunto acotado de eventos y el verificador busca una postcondición. La auditoría debe registrar el recorrido y bloquear nuevas actuaciones si no puede registrar eventos obligatorios.

Las brechas principales descritas son la ausencia demostrada del servidor REST requerido, del flujo visual completo, de las acciones de mouse/teclado asociadas a capturas vigentes, de una integración estricta del presupuesto y deadline, y de la verificación final de negocio. También existen problemas en la vinculación de observaciones, referencias DOM, autorización contextual y manejo de resultados inciertos.

2. Requisitos y fronteras del reto

El sistema debe operar aplicaciones desconocidas sin API ni documentación, usando el acceso a escritorio suministrado en la evaluación. La instrucción funcional se expresa en lenguaje natural y se acompaña de las credenciales o parámetros del escritorio. No debe depender de scripts preparados específicamente para cada aplicación evaluada.

El contrato y las condiciones citadas en el archivo de origen son requisitos a verificar contra el README oficial; no deben considerarse definitivos si el contrato oficial define nombres, campos o semánticas diferentes.

- **REST:** GET /health, POST /tasks y GET /tasks/{task_id}; Bearer obligatorio donde lo exija el contrato.
- **Admisión de tarea:** POST /tasks debe devolver task_id en menos de 2 segundos, sin esperar el ciclo completo de ejecución.
- **Deadline:** máximo de 300 segundos por tarea, incluyendo cola, inferencias, acciones, recuperación y cierre dentro del presupuesto operativo.
- **Operación:** publicar el servicio desde las 20:00 hasta que finalice el Demo Day, según la instrucción descrita en la fuente.
- **Generalización:** evitar reglas por aplicación, selectores predefinidos para cada caso o conocimiento fijo de la disposición de la interfaz.
- **Seguridad:** no permitir que una instrucción observada dentro de la app incremente privilegios ni que el modelo ejecute código arbitrario.

Frontera de alcance

La solución administra la sesión y automatiza la interfaz que el operador está autorizado a usar. No sustituye controles de acceso de la aplicación externa ni garantiza reversión de operaciones remotas. Un timeout, una caída de proceso o un fallo de auditoría no revierten efectos de negocio que ya hayan ocurrido.

3. Estado de implementación observado

La tabla resume la evidencia técnica descrita en el archivo suministrado. “Existente” significa que la fuente identifica el módulo o mecanismo en el repositorio compartido; no significa que haya pasado una prueba funcional independiente.

Componente / ruta citada	Capacidad observada	Estado y limitación relevante
core/schemas.py	Modelos Pydantic, perfiles, límites, permisos, controles autorizados, condiciones finales y acciones DOM.	Existente; la superficie de acciones no cubre aún todas las operaciones visuales objetivo y la vinculación contextual necesita fortalecerse.
core/runner.py	Bucle de propuesta y actuación, timeout, auditoría obligatoria y cierre de sesión.	Base existente; requiere observaciones visuales, deadline externo estricto, estado de snapshot consistente y verificación más fuerte.
browser/session.py	Sesiones Chromium con contexto independiente, restricciones de red y cierre acotado.	Existente; bloquea WebSockets, lo que puede impedir escritorios remotos como noVNC/Guacamole. BrowserContext no es aislamiento de proceso.
browser/observe.py	Extracción de HTML limitada a controles.	Existente; roles, unicidad y vigencia de referencias requieren correcciones.
openrouter.py	Adaptador de inferencia basado en petición textual.	Existente como inferencia textual; no envía imágenes, por lo que no implementa percepción VLM visual.
scripted.py	Propuestas simuladas para pruebas.	Existente; útil en pruebas deterministas, no prueba capacidades reales de generalización.
audit/events.py	Registro JSONL con redacción.	Existente; no aporta cadena criptográfica ni almacenamiento inmutable.
budgets.py	Reservas y confirmaciones de presupuesto.	Parcial; integración opcional, límites de tokens incompletos y coste basado en tokens no equivale a facturación real.
recovery/strategies.py	Clasificación de estrategias de recuperación.	Parcial; la fuente dice que no está integrada en el bucle principal.
entrypoint.py	Adaptador local y comando de salud.	Existente; no es el servidor REST requerido.
Visión, skills y despliegue	Componentes descritos como dirección de diseño.	Pendientes o no demostrados en los archivos resumidos por la fuente.
MCP propio	Servidor con sesión/políticas parcialmente paralelas y operaciones incompletas.	No necesario en el MVP; si se mantiene, debe delegar en el mismo núcleo y no duplicar políticas.

Implicación de diseño

Antes de añadir capacidades nuevas conviene unificar contratos y políticas compartidas: los adaptadores (DOM, visual, API y MCP) deben converger en el mismo runner, el mismo autorizador y el mismo ejecutor. Mantener bucles paralelos crea riesgo de divergencias de seguridad y estados difíciles de auditar.

4. Arquitectura modular objetivo

Se propone un servicio modular en un único despliegue lógico. No existe una necesidad demostrada de dividir la aplicación en microservicios: el reto principal es preservar una frontera de control única, latencia baja y comportamiento cancelable. La API gestiona autenticación, validación, admisión y consulta de estado; un supervisor administra deadlines y workers; cada tarea obtiene una sesión aislada y recursos limitados.

La arquitectura separa plano de control y plano de ejecución. El plano de control recibe solicitudes, fija límites, asigna task_id y expone estados. El plano de ejecución observa la interfaz, solicita una propuesta, la valida, opera la sesión y verifica el resultado. Auditoría, presupuesto y política son controles transversales, no pasos opcionales.

Flujo principal

```
Evaluador -> API autenticada -> registro de tareas -> supervisor/worker -> AgentRunner -> observar
-> proponer -> validar -> ejecutar -> verificar -> resumir y cerrar. El ciclo de ejecución vuelve a
observar después de acciones que puedan cambiar la interfaz. La recuperación no puede saltarse la
política ni reintentar ciegamente una acción persistente.
```

Módulo lógico	Responsabilidad	Contrato de salida esperado
API REST y autenticación	Validar token, payload, límites y ruta; registrar tarea y responder rápidamente.	task_id o error de contrato; sin ejecución síncrona de la tarea.
Task registry + supervisor	Mantener estado público, deadline monotónico, cancelación y aislamiento del worker.	Estado consultable, finalización dentro del plazo o resultado incierto explícito.
AgentRunner	Coordinar la máquina de estados y aplicar límite de pasos, tiempo y transiciones.	RunResult con estado, resumen, evidencia y motivo de bloqueo/fallo.
Observadores	Crear una observación visual y/o DOM, con ID y metadatos de vigencia.	Observation versionada; geometría y contexto suficientes para revalidar acciones.
Adaptador LLM/VLM	Interpretar objetivo y observación, devolver una propuesta estructurada.	Propuesta no confiable, vinculada a la observación y sin acceso directo a herramientas.
Política y validador	Comprobar esquema, contexto, objetivo, riesgo y alcance permitido.	Autorización explícita o rechazo con motivo auditable.
Skills + ejecutor	Traducir acciones autorizadas en eventos Playwright acotados.	Resultado de ejecución, evidencias y error tipificado.
Verificador/recuperación	Comprobar pre/postcondiciones y decidir continuar, reparar o finalizar.	verificado, no_verificado o incierto; nunca éxito por una señal aislada.
Auditoría + presupuesto	Registrar hitos y limitar consumo de tiempo/modelo.	Eventos correlacionados por task_id/run_id/observation_id y medición de recursos.

5. Percepción dual: visión y DOM

Modo visual (visual_only). Es la ruta principal para la evaluación de GUI/canvas. El agente observa una captura del viewport, la interpreta y propone coordenadas normalizadas y operaciones de entrada. No inspecciona el DOM de la aplicación como parte de la percepción visual obligatoria. Este modo debe seleccionarse explícitamente; no depender únicamente de heurísticas de URL o detección de canvas.

Modo DOM. La observación estructural puede producir roles, nombres, valores y referencias temporales verificables cuando su uso está permitido. La base actual descrita ya extrae controles, pero la corrección de roles, la comprobación real de unicidad y la vigencia del snapshot necesitan reforzarse.

Modo auto. Solo es apropiado cuando ambos modos están autorizados. No debe activar inspección DOM en una evaluación visual obligatoria por el simple hecho de que DOM resulte más fácil.

- Una URL con “vnc” o una gran superficie canvas pueden ser indicios; no prueban por sí solas las restricciones de modalidad.
- La exploración debe orientarse al objetivo: observar rutas, menús, resultados y errores; las hipótesis son revisables y la memoria espacial se limita a la tarea.
- Una referencia DOM o coordenada previa no debe sobrevivir a cambios de página, viewport o geometría sin revalidación.
- Los screenshots y el texto visible constituyen datos no confiables: las instrucciones que aparezcan en la app no pueden modificar políticas ni permisos.

6. Modelo de observación y geometría

Cada observación visual debe ser un objeto estructurado, no solamente una cadena de texto. Como mínimo debe identificar observation_id, timestamp, sesión/página, dimensiones del viewport en píxeles CSS, dimensiones de la imagen capturada, versión geométrica y transformaciones aplicadas. Si se recorta o escala la imagen enviada al modelo, deben conservarse los parámetros de transformación hacia el viewport original.

El MVP debe capturar el viewport completo, preferentemente en PNG. Una captura de página completa no es una referencia válida para clicar elementos situados fuera del viewport visible: la coordenada devuelta debe mapearse a la superficie que realmente recibe los eventos de entrada.

Playwright opera con coordenadas CSS. La densidad de píxel (DPR) del PNG no debe confundirse con la geometría del cursor. Si el modelo entrega coordenadas normalizadas enteras entre 0 y 1000, el mapeo propuesto es el siguiente:

```
x_css = round((x_norm / 1000) * (W_css - 1))
y_css = round((y_norm / 1000) * (H_css - 1))
Invariantes: 0 <= x_css < W_css; 0 <= y_css < H_css
```

Antes de ejecutar el clic, el núcleo debe verificar que el observation_id siga vigente, que la página y sesión correspondan, que las dimensiones coincidan y que el punto se encuentre dentro de la superficie autorizada. Si cualquiera de estas condiciones falla, se rechaza la acción y se vuelve a observar. Estos controles reducen carreras de interfaz, pero no garantizan que la pantalla permanezca inmóvil entre la validación y el evento.

Campo de observación propuesto	Propósito
observation_id	Correlacionar propuesta, acción, auditoría y verificación con una instantánea concreta.
timestamp / session_id / page_id	Detectar observación antigua, cambio de sesión o navegación.
viewport_width_css / viewport_height_css	Definir límites y sistema de coordenadas que recibe Playwright.
image_width_px / image_height_px / format	Registrar dimensiones reales de la evidencia visual enviada al modelo.
geometry_version / transforms	Hacer reproducible la conversión si hubo resize, crop o reescalado.
visual_reference	Referencia controlada a la captura; no implica que el archivo quede expuesto al usuario o al proveedor.
dom_snapshot_id (si aplica)	Vincular referencias DOM a una observación concreta y temporal.

7. Contrato de acciones y contexto de ejecución

El esquema actual descrito admite navigate, click, fill, submit, extract, done y ask_human. Estas operaciones no bastan para automatizar GUI pura; el diseño debe extenderse con clic espacial, doble clic, clic secundario, escritura por teclado, atajos, scroll y espera. Drag-and-drop solo se añade si el alcance lo requiere. Añadir una operación al esquema no basta: deben definirse precondiciones, límites, semántica de fallo, política de autorización y postcondición verificable.

Toda acción que dependa del estado de la interfaz debe incorporar observation_id. El núcleo revalida la propuesta aunque el proveedor use JSON estructurado o garantice un esquema de salida: el formato correcto no hace que una acción sea segura ni correcta.

El adaptador de modelo debe recibir un TaskContext y una Observation (o una estructura equivalente) en vez de un objective y un page_state textual sin identidad. El contexto delimita el objetivo, el modo de percepción, el presupuesto restante, el alcance de operaciones y las condiciones finales.

Acción	Validación previa	Evidencia posterior
Clic espacial	Snapshot vigente; x/y dentro de viewport; acción acorde al objetivo y al alcance autorizado.	Nueva observación y cambio esperado; no asumir éxito por el evento enviado.
Escritura / fill	Foco correcto; campo identificado; texto o referencia de secreto autorizada; política de reemplazo explícita.	Valor visible o estado equivalente confirmado, sin filtrar secreto en logs.
Atajo / teclado	Combinación permitida, foco y efecto esperado; liberar modificadores en limpieza.	Cambio de estado coherente con la acción, o resultado incierto.
Scroll / espera	Magnitud, duración y número de repeticiones acotados; sin exceder deadline.	Viewport actualizado o condición de espera satisfecha.
Submit / operación persistente	Formulario y destino correctos; consecuencias dentro del alcance permitido.	Postcondición funcional; ante timeout consultar resultado antes de repetir.
DONE	No es acción de éxito; solicita al verificador evaluar condiciones finales.	Éxito solo si las condiciones objetivas quedan respaldadas por evidencia.

Acción	Validación previa	Evidencia posterior
ASK_HUMAN	Solo se utiliza si existe un canal de intervención habilitado.	Bloqueo seguro si no existe canal o no se recibe la aprobación requerida.

8. Runner, estados y ejecución determinista

El AgentRunner coordina la máquina de estados. El ciclo base es observar, proponer, validar, ejecutar, verificar y decidir si continuar, recuperar o finalizar. Debe existir un único camino para ejecutar acciones autorizadas; ni las skills ni los adaptadores alternativos pueden actuar directamente sobre la sesión sin pasar por el mismo autorizador y auditoría.

“Determinista” describe la traducción de una propuesta validada en eventos explícitos, acotados y auditables; no garantiza que el sistema remoto ni el modelo produzcan siempre el mismo resultado. Para acciones que alteran la interfaz, se recomiendan puntos de observación entre pasos en vez de cadenas largas de eventos invisibles.

El límite duro de cinco minutos debe imponerse por la capa supervisora, aunque el perfil interno solicite un tiempo mayor. La fuente indica que el perfil puede permitir hasta 3600 segundos, por lo que el contrato de evaluación tiene que prevalecer sin depender de la configuración aportada por la tarea.

- El runner comprueba cada transición, número de pasos y tiempo restante antes de emitir una acción.
- Una acción fallida genera observación y diagnóstico antes de escoger una alternativa autorizada.
- No hay reintento automático de violaciones de política.
- El resultado extract debe incorporarse efectivamente al RunResult; la fuente identifica que esto no sucede de forma completa en el flujo actual.
- Cierre de navegador, emisión de auditoría y limpieza tienen plazos acotados para no dejar el estado público fuera del deadline.

9. Skills nativas

Las skills son procedimientos reutilizables de nivel de tarea, no agentes privilegiados. Deben usar las mismas interfaces de observación, política, ejecución y verificación que el resto del sistema. No conceden acceso general al filesystem, terminal, página de Playwright ni ejecución arbitraria de código.

Skill propuesta	Responsabilidad	Restricción técnica
DesktopAuthAndBootstrap	Conectar al escritorio y completar autenticación inicial.	Los secretos se resuelven en ejecución; nunca se incorporan al prompt ni a la auditoría. Maximizar/ajustar ventana solo si está autorizado.
CanvasFormFill	Localizar foco y completar campos en canvas o GUI.	No borrar valores por defecto; reemplazo explícito; cadencia de escritura probada y comprobación posterior.
LegacyPopupManager	Interpretar diálogos y consecuencias.	No aceptar automáticamente por el aspecto del botón o por ser la opción predeterminada.
ReportExportTracker	Navegar menús y confirmar exportación/guardado.	Distinguir archivo remoto de descarga del navegador; no inventar nombre, tamaño ni contenido.
ZeroShotDiscovery	Explorar rutas y módulos desconocidos.	Mantener hipótesis revisables; no asumir distribución de menús universal ni persistir coordenadas de forma indefinida.
SelfHealingRetry	Reobservar, diagnosticar y probar una alternativa.	Comprobar si una operación persistente ya ocurrió; no encadenar clic, doble clic y Escape genéricamente.

10. Política, autorización y fronteras de confianza

La política convierte el permiso abstracto para realizar una tarea en autorización contextual de una operación concreta. En modo DOM, la decisión debe asociar página, operación, rol/nombre, entrada, ámbito y postcondición. El control actual descrito comprueba existencia de controles en varios casos, pero no vincula de manera consistente la autorización con el control específico sobre el que se actuará.

En modo visual, un inventario previo de botones no es compatible con una aplicación desconocida. La autorización se expresa mediante el objetivo de tarea, el escritorio autorizado, las clases de operación y los límites de impacto. Un punto dentro del viewport no demuestra autorización de negocio; Pydantic y geometría validan aspectos mecánicos, no clasifican infaliblemente consecuencias destructivas.

La pantalla, el DOM, los nombres de archivos y cualquier texto de la aplicación se tratan como contenido no confiable. Las instrucciones inyectadas en una aplicación no amplían permisos, no desactivan restricciones y no habilitan herramientas. Las credenciales se resuelven dentro de la capa de ejecución mediante referencias seguras, sin exponerlas al modelo.

- La política R3 indicada en la fuente exige intervención humana; si no hay un canal de aprobación dentro del contrato, la acción que requiere aprobación debe quedar bloqueada.
- La política debe ser fail-closed ante permisos ausentes, contexto incoherente, auditoría no disponible o token inválido.
- Debe distinguirse la validez sintáctica de una propuesta de su autorización semántica.
- Los eventos potencialmente persistentes requieren controles proporcionales a su impacto y evidencia explícita de finalización.

11. Red, sesiones y escritorios remotos

El módulo de sesión descrito bloquea WebSockets, lo que puede impedir transportes remotos como noVNC o Guacamole. La corrección no es habilitar cualquier WS/WSS, sino autorizar endpoints concretos del escritorio suministrado y mantener restricciones para destinos no relacionados. La conectividad de la aplicación remota, la del cliente, los puertos, los orígenes y los destinos deben modelarse como elementos separados.

La validación HTTP actual no constituye por sí misma un validador exhaustivo de WebSockets. Una política de proxy obligatorio aparece descrita, pero no se aplica uniformemente en el camino real. La arquitectura requiere una sola política de red aplicada de forma consistente por sesión y por infraestructura.

Si el acceso oficial requiere un destino privado concreto, la excepción debe ser explícita y estrecha; nunca se debe desbloquear una red privada completa a partir de una URL arbitraria. BrowserContext permite separar cookies y sesiones de páginas, pero no sustituye aislamiento de proceso, contenedor, filesystem ni red.

- Usar allowlists explícitas de host, esquema, puerto y ruta cuando proceda.
- Evitar redirecciones que eludan validación de destino y validar también el destino efectivo después de resolver redirecciones.
- Mantener timeouts de conexión y lectura; cerrar sesiones y liberar recursos al terminar.
- Probar el canal WebSocket requerido por el entorno de evaluación sin abrir destinos adicionales.

12. Verificación y recuperación

La verificación debe basarse en postcondiciones observables. La diferencia visual entre capturas es una señal de diagnóstico, no una prueba autónoma de éxito: puede haber cambios irrelevantes o no existir un cambio visible aunque una operación haya terminado. En DOM, las condiciones también deben verificar el resultado de negocio, no solo que se disparó un evento o terminó una navegación.

La verificación final compara la solicitud original con evidencia del estado resultante: dato persistido, búsqueda correcta, registro actualizado o exportación efectivamente completada. En GUI pura puede apoyarse en un VLM; debe declarar la fuerza y tipo de evidencia. Dos evaluaciones visuales correlacionadas no eliminan el riesgo de falso positivo.

Un timeout posterior a una operación de guardado no significa que la operación haya fallado. Antes de repetir una operación persistente, la recuperación comprueba si ya se produjo mediante la interfaz autorizada. Si no puede

determinarlo, el resultado correcto es incierto/no verificado, no éxito inventado ni reintento ciego.

Estados de verificación recomendados

Estado	Semántica	Comportamiento esperado
verificado	Existe evidencia suficiente de que la postcondición del objetivo se cumplió.	Finalizar con éxito y adjuntar resumen/evidencia mínima protegida.
no_verificado	La evidencia disponible no respalda el resultado solicitado.	No declarar éxito; puede continuar solo si quedan tiempo y acciones autorizadas.
incierto	No puede determinarse si la operación produjo efecto, por ejemplo tras timeout.	No repetir operaciones persistentes sin comprobar antes el estado remoto; informar incertidumbre.

Secuencia de recuperación

```
Reobservar -> clasificar el fallo -> comprobar posibles efectos -> seleccionar alternativa permitida -> ejecutar una acción acotada -> verificar. La recuperación debe conservar el motivo original y el historial de intentos; nunca debe convertir un bloqueo de política en otra vía de ejecución.
```

13. API REST, estados públicos y deadlines

El entrypoint descrito es local y no constituye el servidor de evaluación. Se recomienda un servidor HTTP autenticado (FastAPI/ASGI aparece como opción de diseño, no como dependencia ya confirmada), con registro de tareas y supervisor de workers. El endpoint de creación debe autenticar y validar, registrar la tarea y fijar el deadline antes de devolver task_id; no debe esperar a Chromium o al proveedor de modelos.

El deadline es monotónico y compartido por runner, skills, adaptadores de modelo, verificador y limpieza. El plazo máximo de la evaluación es 300 segundos y debe incluir el tiempo de cola. La cancelación cooperativa se complementa con cierre acotado y terminación del worker si no responde. Detener un worker no revierte operaciones que ya se enviaron a la aplicación externa.

GET /health debe comprobar readiness de dependencias y el estado conocido del servicio, sin requerir una inferencia facturada en cada consulta. GET /tasks/{task_id} expone estado y resumen sin filtrar secretos ni evidencias sensibles.

Ruta / elemento	Contrato y controles esperados
POST /tasks	Bearer; validación del payload; límites; creación de task_id en < 2 s; encolar/despachar sin ejecutar la tarea de forma síncrona.
GET /health	Readiness real, estado de dependencias y capacidad de aceptar trabajo. Respuesta barata y no dependiente de inferencia.
GET /tasks/{task_id}	Estado, resumen, error/block reason y timestamps; autorización para consultar la tarea; respuesta controlada si el ID no existe.
Supervisor	Deadline absoluto monotónico; máximo 300 s; cancelación y terminación; registro coherente del estado final.
Worker	Proceso/sesión independiente, límites de recursos, cierre de Playwright y salida de resultado serializable.

Mapeo de estado interno a público

Estado interno	Estado público sugerido	Nota
Aceptada / ejecutando	en_curso	Solo si task_id se creó correctamente.
SUCCEEDED_VERIFIED	terminada	Solo tras verificación suficiente; DONE no es prueba.
FAILED	fallida	Añadir error seguro y resumen acotado.
BLOCKED	fallida con motivo de bloqueo	No reintentar violaciones de política.
NEEDS_HUMAN_REVIEW	fallida si no existe canal de intervención	No dejar indefinidamente una tarea esperando una capacidad que no existe.
Deadline excedido / cancelada	fallida o estado de timeout establecido por el contrato oficial	El resultado puede ser incierto si se enviaron acciones persistentes.

14. Auditoría, evidencia y secretos

Los eventos deben distinguir propuesta, decisión de política, acción aceptada, ejecución, verificación, recuperación, cancelación y estado final. Deben correlacionarse con task_id, run_id y observation_id para poder reconstruir el razonamiento operacional sin exponer contenido sensible. El JSONL citado ya incorpora redacción y validación de rutas, pero la fuente señala que no es criptográfico ni inmutable.

Una cadena de hashes futura podría ayudar a detectar modificaciones bajo supuestos concretos; no garantiza inmutabilidad absoluta si el mismo operador puede cambiar logs y claves. Se requiere definir almacenamiento, retención, control de acceso y protección para capturas y trazas, además de los archivos de eventos.

La auditoría obligatoria debe fallar cerrada. Aun así, si la escritura del evento falla después de haber actuado, la aplicación remota podría haber recibido el evento: no existe transacción atómica que incluya simultáneamente el log local y la acción en el sistema remoto. El resultado debe registrarse como potencialmente incierto y el runner no debe continuar sin los controles requeridos.

Los secretos se resuelven en tiempo de ejecución, no mediante texto de prompt. Los datos funcionales autorizados deben separarse de su representación redactada: enmascarar un correo o identificador en la auditoría no puede cambiar silenciosamente el valor que debe introducirse en la aplicación.

- Redactar credenciales, tokens, datos personales y valores sensibles en prompts, logs, errores y resúmenes.
- Aplicar permisos mínimos a carpetas de evidencia y establecer política de expiración para screenshots.
- Registrar identificadores y referencias de secretos, no el secreto en claro.
- Incluir motivo de rechazo y postcondición verificada, evitando almacenar contenido visual completo si no es necesario.

15. Inferencia, proveedores y presupuestos

La orquestación, las skills, la ejecución y la verificación deben seguir siendo propiedad del núcleo del agente. Playwright, Pydantic y HTTPX proporcionan primitivas; no sustituyen las políticas del producto. OpenRouter aparece como integración opcional. El proveedor interpreta observaciones y devuelve propuestas, pero no debe recibir acceso directo al escritorio, credenciales ni herramientas de ejecución.

La selección de proveedor/modelo debe considerar soporte real de visión, latencia, disponibilidad, costo, límites de salida y tratamiento de datos. Cualquier uso de proveedor externo queda condicionado a las reglas oficiales de la evaluación. El servidor MCP externo de archivos descrito en la configuración se excluye del camino del MVP para reducir superficie y dependencias.

El presupuesto debe ser obligatorio para llamadas reales: límites de cantidad de llamadas, tokens de entrada/salida, resolución y tamaño de imágenes, tiempo y costo estimado. El registro de una tarifa fija por total_tokens no representa necesariamente el costo facturado, especialmente cuando varían los precios por modelo, modalidad y proveedor.

- La CLI descrita no inyecta BudgetManager en el flujo real de forma obligatoria.

- max_output_tokens se registra pero la imposición completa está pendiente.
- Concurrencia, reintentos, restauración de estado y llamadas a modelos deben consumir presupuesto compartido.
- Fijar versiones mediante lockfile y documentar navegador, entorno de ejecución y variables de configuración.
- El archivo fuente no incluye lockfile ni Dockerfile; reproducibilidad y despliegue no están demostrados.

16. Brechas y defectos específicos

La lista siguiente conserva los hallazgos concretos enumerados en el material aportado. Deben convertirse en issues rastreables, corregirse en código y cerrarse con pruebas; no basta con que la documentación describa la solución.

Prioridad sugerida	Hallazgo reportado	Corrección / evidencia de cierre
P0	observation_id es opcional y no se compara con la observación actual.	Exigir ID en acciones dependientes de estado; rechazar mismatch y probar navegación/cambio de viewport.
P0	El runner reconstruye PageObservation sin conservar ID del snapshot.	Propagar identidad y geometría de forma inmutable desde percepción hasta acción/auditoría.
P0	El perfil permite hasta 3600 s; el reto establece 300 s.	Aplicar deadline independiente en supervisor; prueba de timeout en todos los puntos de espera.
P0	No se demuestra servidor REST de evaluación.	Implementar Bearer, POST rápido, health, consulta de task y estados; pruebas de contrato.
P0	Percepción visual y acciones espaciales completas están pendientes.	Captura de viewport, VLM, mouse/teclado/scroll, geometría y vínculo con snapshot; prueba GUI pura.
P1	password_field no cumple patrón de role; roles extraídos pueden no corresponder con roles accesibles Playwright.	Normalizar esquema/roles y testear cada tipo de referencia.
P1	unique=True se asigna sin comprobar unicidad.	Comprobar cardinalidad real y rechazar referencias ambiguas.
P1	La autorización de click/fill no siempre vincula el objetivo al control concreto.	Evaluar página, referencia, operación, entrada, ámbito y postcondición.
P1	click/submit se consideran verificados tras esperar carga, sin postcondición de negocio sistemática.	Separar ejecución de verificación y probar casos de falso éxito.
P1	extract no se incorpora completamente a RunResult.	Propagar datos extraídos con esquema y redacción consistente.
P1	Cierre y auditoría pueden seguir después del timeout.	Deadline del supervisor determina estado público; limpieza acotada y worker terminable.
P1	WebSockets bloqueados y proxy/política de red no se aplican uniformemente.	Allowlist WS/WSS y política única por sesión; pruebas negativas de SSRF/destinos no autorizados.
P1	Recovery strategies no está integrada en el bucle principal.	Integrar recuperación con límites y reobservación; probar que políticas no se reintentan.
P2	BudgetManager opcional; coste y max_output_tokens no se imponen completamente.	Inyección obligatoria, límites compartidos y prueba de consumo/cancelación.
P2	MCP mantiene sesión y políticas paralelas y operaciones parciales.	Excluir del MVP o convertirlo en adaptador del núcleo común.
P2	No se demuestra lockfile/Dockerfile ni despliegue reproducible.	Añadir lockfile/imagen de ejecución y ejecutar instalación limpia en CI.

17. Plan de construcción por fases

El orden de trabajo propuesto prioriza una ruta vertical funcional y segura antes de incorporar abstracciones complejas. Cada fase debe terminar en una demostración ejecutable y un criterio verificable; las capacidades no comprobadas permanecen marcadas como pendientes.

Fase	Entregables	Puerta de salida
0. Congelar contrato	Contrastar README oficial; fijar campos de tarea, auth, estados, plazo, payload y formato de resumen.	Tests de contrato versionados; discrepancias resueltas.
1. Endurecer núcleo	Snapshot IDs, roles DOM, unicidad, validación cruzada, RunResult y auditoría correlacionada.	Tests unitarios de acciones inválidas y stale observation.
2. Conectar escritorio	Sesión remota, allowlist de destino, transporte WS/WSS explícito y aislamiento.	Conexión del escritorio autorizada; destinos no autorizados bloqueados.
3. Captura + geometría	Captura de viewport, Observation schema, normalización de coordenadas, transformaciones.	Tests en múltiples resoluciones/DPR, bordes y cambios de página.
4. Inferencia visual	Adaptador VLM configurable; imagen + objetivo + historial acotado; salida tipada.	Propuestas estructuradas no ejecutables hasta validar; límites de tamaño/tiempo.
5. Ejecutor GUI	Click espacial, teclado, escritura, atajos, scroll y waits con cleanup.	Formulario y diálogo operables sin inspección DOM de la app.
6. Política + skills	Pre/postcondiciones y skills DesktopAuth, CanvasFormFill, Popup, Export, Discovery, Retry.	Cada skill pasa por el ejecutor común; violaciones bloqueadas.
7. Verificación y recuperación	Estados verificado/no_verificado/incierto, reobservación y prevención de dobles guardados.	No hay falso éxito al usar DONE o pixel-diff; persistencia incierta no se repite ciegamente.
8. REST + supervisor	API autenticada, task registry, worker, 300 s monotónicos, cancelación y resumen.	POST < 2 s; consulta de estado; timeout efectivo; casos de concurrencia.
9. Operación/release	BudgetManager obligatorio, lockfile, despliegue reproducible, logs, readiness y ensayo del Demo Day.	Instalación limpia y ventana de disponibilidad probadas.

18. Estrategia de pruebas y criterios de aceptación

Las pruebas deben medir por separado validez de contrato, seguridad, finalización verificada, falsos éxitos, latencia y generalización. Un test que únicamente comprueba que se ejecutó click() no certifica la operación real. El conjunto debe incluir aplicaciones o flujos no usados para ajustar el agente y casos adversariales de contexto obsoleto, bloqueo y caída de dependencia.

Área	Prueba	Criterio de aceptación
Geometría	Distintas resoluciones/DPR, límites, crop/resize, scroll y cambio de página.	No hay clics fuera de viewport; observaciones obsoletas y geometría incompatible se rechazan.
GUI pura	Completar formulario y manejar diálogo sin consultar DOM de la aplicación.	Éxito respaldado por postcondición, no por evento enviado.
Red	Conectar por transporte autorizado; intentar destinos privados/externos no permitidos.	El escritorio funciona y los demás destinos permanecen bloqueados.
Riesgo/política	Acción fuera del objetivo, botón destructivo, prompt injection y permiso faltante.	Bloqueo pre-ejecución con motivo auditado; ningún adaptador alterno evade la regla.
REST	Token ausente/inválido, payload defectuoso, task desconocida, concurrencia.	Contrato consistente, errores controlados y POST bajo 2 s.
Deadline	Modelo lento, navegador bloqueado, cierre defectuoso y tarea en cola.	No se inician acciones después de 300 s; estado final publicado por supervisor.
Auditoría/privacidad	Secretos en entrada, excepciones, capturas y logs; fallo de escritura del log.	No hay secreto en evidencia; el sistema se detiene o marca incertidumbre según el punto de fallo.
Verificación	DONE prematuro, pixel-diff vacío, guardado con timeout y exportación remota.	Solo se declara éxito cuando la postcondición se prueba; efecto incierto se comunica.
Generalización	Aplicaciones y layouts no vistos durante desarrollo.	Métricas independientes de éxito verificado y falsos positivos; fallos clasificados y reproducibles.

Métricas mínimas para reportar

- Tasa de tareas terminadas y verificadas, separada de tareas simplemente ejecutadas.
- Tasa de falso éxito (el agente declara éxito sin postcondición confirmada).
- Tasa de bloqueo correcto de acciones no autorizadas.
- Latencia de admisión REST, tiempo total de tarea y distribución de consumo por proveedor.
- Número de recuperaciones, acciones persistentes inciertas y errores atribuibles a geometría, inferencia, red o política.

19. Decisiones de arquitectura (ADR)

ADR-001 - Visión como modalidad principal. El sistema debe operar GUI/canvas sin depender de DOM cuando el entorno de evaluación lo exija. DOM se conserva como modalidad opcional explícitamente habilitada. Las acciones se vinculan a un observation_id y a geometría vigente.

ADR-002 - Propuesta separada de ejecución. El modelo genera propuestas; el código propio valida, autoriza, ejecuta y verifica. Ninguna skill o adaptador proporciona una ruta de ejecución paralela que omita controles.

ADR-003 - Núcleo común y proveedores intercambiables. El modelo de inferencia se trata como dependencia externa intercambiable. El proveedor no obtiene credenciales ni herramientas. MCP se omite del MVP o funciona como adaptador fino del núcleo compartido.

ADR-004 - Éxito solo por evidencia. DONE, un click aceptado, una espera de navegación o una diferencia de píxeles no definen éxito. El estado final depende de condiciones explícitas y evidencias documentadas.

ADR-005 - Tiempo límite aplicado externamente. El supervisor controla el límite de 300 segundos independientemente del perfil de la tarea; todos los módulos cooperan, pero la garantía final no depende únicamente de cooperación.

20. Checklist operativo de salida

- Contrato oficial revisado y fijado; campos y resumen coinciden con la especificación real.
- Token Bearer validado en rutas protegidas y errores no revelan configuración sensible.
- POST /tasks devuelve task_id antes de comenzar la ejecución larga.
- Deadline monotónico de 300 s cubre cola, modelo, browser, skills, verificación y cancelación.
- La sesión remota conecta solo a endpoints autorizados, incluidos WebSocket si se requiere.
- Modo visual obligatorio evita lectura DOM de la aplicación; modo DOM solo se habilita según política.
- Cada acción depende de una observación vigente; referencia o geometría obsoleta se rechaza.
- Todas las acciones pasan por validación contextual y ejecutor único.
- La verificación final usa postcondiciones de negocio y puede devolver incierto.
- Los secretos no pasan por prompts, logs, trazas ni capturas publicadas.
- La auditoría y los presupuestos están inyectados de forma obligatoria en todas las llamadas reales.
- Las pruebas de contrato, timeout, red, GUI pura, falso éxito y generalización pasan en CI/ensayo.
- La instalación y el despliegue se reproducen desde dependencias fijadas y configuración documentada.

Nota de trazabilidad y limitaciones

Este PDF desarrolla el contenido técnico de la descripción adjunta. El archivo fuente indica que la revisión se realizó estáticamente sobre un repositorio compartido en TXT; el repositorio íntegro, sus pruebas y el entorno de despliegue no fueron ejecutados ni verificados en la generación de este documento. Por ello, los estados “existente”, “parcial” y “pendiente” deben validarse en el código y contra los requisitos oficiales antes de usarse como certificación de entrega.